Motor de Inferencia Lógica para la Clasificación y Sugerencia de Películas

David Alejandro Reyes Pérez

Juan David Collazos Garzon

Programa Ingeniería de Software,

Sistemas Expertos

Juan José Ramos Díaz

Fundación Escuela Tecnológica de Neiva

13 de octubre de 2026

Propósito

Sistema experto que recomienda películas según las preferencias del usuario, su estado de ánimo y el contexto en que las verá. Usa Prolog como motor de inferencia (base de conocimiento y reglas) y Python como interfaz, conectados mediante PySwip.

Casos que cubre

- Recomendación según género, estado de ánimo y compañía (solo, pareja, familia o amigos).
- Filtrado por clasificación de edad, duración máxima, idioma y calificación mínima.
- Sugerencia de películas similares a una de referencia (por género, director o tema).
- Priorización de las mejor valoradas según reseñas almacenadas localmente.
- Explicación de cada recomendación y alternativas cuando no hay coincidencias exactas.

Fuera del alcance

- Aprendizaje automático o historiales de usuarios.
- Consulta de datos, reseñas o disponibilidad en internet o plataformas de streaming.
- Cuentas de usuario, perfiles persistentes o escritura de nuevas reseñas desde el sistema.
- Catálogos extensos: la base contendrá entre 50 y 100 películas.

Entradas

- **Obligatorias:** género, estado de ánimo, compañía, clasificación de edad, duración máxima e idioma.
- **Opcionales:** calificación mínima y película de referencia.

Base de conocimiento

- **Hechos:** datos de cada película (título, género, año, duración, clasificación, idioma, director, temas) y reseñas locales (calificación promedio y etiqueta de opinión) cargadas por los desarrolladores.
- **Reglas:** relacionan el perfil del usuario con las películas. Por ejemplo, si la compañía es la familia se excluyen películas superiores a +12, y si el usuario está estresado se priorizan las cortas y ligeras.

Salidas

Lista de 3 a 5 películas ordenadas por nivel de coincidencia y calificación local, cada una con su justificación y nivel de coincidencia (alto, medio o bajo). Si no hay resultados exactos, se indica qué criterio se flexibilizó.

Limitaciones

Las recomendaciones dependen de la calidad de los datos y de las reseñas locales, que reflejan solo la opinión de la fuente utilizada. Agregar o modificar películas requiere editar manualmente el archivo de Prolog.

1. Contexto y objetivo general

Este segundo entregable evoluciona el prototipo descrito en la propuesta inicial: acopla el motor lógico en Prolog con una aplicación interactiva en Python para que el usuario busque películas, consulte el resultado y audite el razonamiento que lo produjo.

El sistema adopta un enfoque de caja blanca. Cada recomendación conserva su justificación simbólica; las hipótesis que no prosperan incluyen el motivo del descarte, y los empates o coincidencias parciales generan una recomendación de evidencia adicional en lugar de un mensaje genérico de diagnóstico desconocido.

2. División de roles y responsabilidades

• David Reyes — Ingeniero del Conocimiento y Motor Simbólico: estructura en base_conocimiento.pl los predicados para recolectar la traza, identificar hipótesis descartadas y elegir el dato o síntoma adicional con mayor peso para confirmar o desempatar.

• Juan Collazos — Ingeniero de Integración y Explicabilidad: construye el puente Python mediante PySwip, desarrolla la interfaz en main.py y presenta visualmente los módulos «¿Por qué?» y «¿Qué falta?».

La división identifica responsabilidades técnicas complementarias. La integración final mantiene la interfaz, el transporte de datos y la lógica simbólica separados para facilitar su mantenimiento y revisión.

3. Arquitectura híbrida y requisitos técnicos

• Capa de presentación (main.py, Streamlit): captura la búsqueda y los filtros, presenta los resultados y despliega la auditoría. No implementa reglas de selección ni calcula los puntajes del motor.

• Puente (motor.py, PySwip): valida y limpia las entradas, invoca predicados Prolog con argumentos instanciados y recibe la respuesta serializada para la interfaz.

• Capa simbólica (base_conocimiento.pl, SWI-Prolog): normaliza la consulta, aplica hechos y reglas, infiere y ordena hipótesis, construye la traza y calcula decisiones y recomendaciones.

El contrato principal del puente es buscar_json/3: recibe el texto de consulta y la edad (o -1 cuando no se aplica ese filtro) y devuelve una respuesta JSON con estado, criterios, resultados, decisión y auditoría. Python trata el texto como dato y valida tipos y rangos antes de consultar. La lógica del dominio permanece en Prolog.

No se utiliza read/1 en Prolog: la captura ocurre en la interfaz y el motor responde a consultas explícitas, sin bloquear el hilo de la aplicación. Así se conserva un flujo interactivo y se evita mezclar entrada/salida imperativa con inferencia simbólica.

4. Explicabilidad: ¿Por qué?

Para cada resultado, el panel de auditoría presenta la información que permite reconstruir y revisar la inferencia:

• Reglas disparadas: reglas intermedias y finales que sustentan la hipótesis principal, identificadas y descritas para lectura humana.

• Hechos verdaderos: datos del catálogo y evidencia de la consulta que fueron comprobados y unificados durante la inferencia, como el género, una etiqueta o un tema coincidente.

• Hipótesis descartadas: alternativas que compartían parte de la evidencia, pero quedaron fuera por una condición excluyente, por el filtro de edad, por ausencia de evidencia suficiente o por obtener menor puntaje. Se presenta el motivo correspondiente.

La auditoría se construye desde Prolog y acompaña a la respuesta; no se infiere retrospectivamente mediante texto escrito en la interfaz. Esto permite contrastar el resultado con el conocimiento declarado y explicar tanto las coincidencias como las exclusiones.

5. Recomendación: ¿Qué falta?

Cuando varias hipótesis quedan empatadas en primer lugar o la evidencia solo respalda una coincidencia parcial, el motor informa que aún no hay una decisión definitiva. Prolog evalúa qué criterios todavía no se han consultado, cuáles separan a las hipótesis líderes y qué prioridad tienen; con esa evaluación formula la pregunta adicional más útil.

La interfaz comunica claramente el dato o preferencia solicitado y las hipótesis que compiten. Por ejemplo, ante varias películas divertidas con el mismo puntaje, puede preguntar si el usuario necesita una opción apta para público infantil. La respuesta a esa pregunta permite confirmar o separar candidatas, en vez de limitarse a diagnóstico desconocido.

6. Escenarios de demostración y validación

• Decisión definitiva y trazable: la búsqueda películas de romance propone Titanic como hipótesis única y permite revisar las reglas activadas y los hechos verdaderos.

• Evidencia insuficiente y desempate: la búsqueda películas divertidas produce varias hipótesis líderes; el sistema declara el empate y solicita un criterio adicional, como la aptitud para público infantil.

• Condición excluyente: en la búsqueda infantil, el motor aplica las restricciones declaradas y muestra en la auditoría por qué una hipótesis incompatible fue descartada.

• Validación del puente: las pruebas automatizadas cubren consultas vacías o desconocidas, filtros de edad, normalización, deduplicación, entrada segura, explicaciones, desempates y otros flujos de integración.

7. Artefactos del segundo entregable

• main.py: interfaz web interactiva desarrollada con Streamlit.

• base_conocimiento.pl: catálogo, reglas de inferencia, trazabilidad y recomendación de evidencia adicional.

• README.md: requisitos, instalación, ejecución y casos sugeridos para demostrar la aplicación.

• DICCIONARIO_PREDICADOS.md: documentación técnica de predicados y sus modos de uso.

• motor.py: puente PySwip que valida entradas y transporta la solicitud y respuesta entre Python y Prolog.

• pruebas.py y requirements.txt: pruebas reproducibles y dependencias del proyecto.

8. Actualización del diccionario de predicados

El diccionario técnico documenta el nombre y aridad, los argumentos de entrada (+), salida (-) o uso bidireccional (?), y la función de los predicados nuevos. Entre los predicados centrales se encuentran:

• reglas_disparadas/3 y hechos_verdaderos/4: reconstruyen las reglas activadas y los hechos comprobados para una hipótesis.

• hipotesis_descartadas/5 y auditoria_motor/5: explican las alternativas descartadas y agrupan la información de auditoría.

• recomendacion_desempate/3, recomendacion_confirmacion/3 y decision_motor/4: evalúan la evidencia insuficiente, proponen una pregunta discriminante y determinan el estado de la decisión.

• buscar_json/3: expone a la aplicación la búsqueda, el dictamen, la recomendación y la auditoría en un contrato JSON.

9. Consideraciones y cierre

La idea ofrece una experiencia interactiva y conserva la separación entre presentación, integración e inferencia. La explicación y la recomendación de evidencia proceden del motor lógico, de modo que el usuario puede inspeccionar el fundamento de una propuesta y saber qué información falta para resolver una ambigüedad.

El catálogo y las etiquetas son didácticos y locales; las edades incluidas no sustituyen clasificaciones oficiales. Las opiniones de ejemplo se utilizan para demostrar reglas y no representan evaluaciones verificadas de fuentes externas.